

# Window Functions

Advanced Analytics Without Losing Row Detail

## ■ LEARNING OUTCOME

averages,  
P BY.

### ■ Skills You Will Gain

- Use ROW\_NUMBER, RANK, DENSE\_RANK for ranking
- Calculate running totals with SUM OVER
- Compute moving averages with AVG OVER
- Access previous and next rows with LAG and LEAD
- Use FIRST\_VALUE, LAST\_VALUE, NTH\_VALUE
- Apply PARTITION BY and ORDER BY inside OVER

■ Window functions are the most powerful feature in modern SQL. They enable ranking, running totals, moving averages, and period comparisons -- all in one SELECT without GROUP BY collapsing your data.

## SECTION 1

# Window Function Anatomy

Syntax: function() OVER (PARTITION BY ... ORDER BY ... ROWS/RANGE ...)

PARTITION BY: divides rows into groups (like GROUP BY but keeps all rows) ORDER BY: defines the order within each partition ROWS/RANGE: defines the window frame (which rows are included in calculation) Key difference from GROUP BY: GROUP BY COLLAPSES rows -- one output row per group WINDOW FUNCTIONS KEEP all rows -- each row gets its own calculation Window functions execute AFTER WHERE, GROUP BY, and HAVING but BEFORE ORDER BY and LIMIT.

## SECTION 2

# Ranking Functions

```
-- ROW_NUMBER: unique sequential number (no ties) -- RANK: same number for ties, then skips numbers
-- DENSE_RANK: same number for ties, no skipping
SELECT customer_id, annual_spend, ROW_NUMBER()
OVER (ORDER BY annual_spend DESC) AS row_num, RANK() OVER (ORDER BY annual_spend DESC) AS rank_pos,
DENSE_RANK() OVER (ORDER BY annual_spend DESC) AS dense_rank FROM customers; -- Example output: --
spend=90000: row_num=1, rank=1, dense_rank=1 -- spend=85000: row_num=2, rank=2, dense_rank=2 (same
as row_num) -- spend=85000: row_num=3, rank=2, dense_rank=2 (tie handled differently) --
spend=70000: row_num=4, rank=4, dense_rank=3 (rank skips 3, dense_rank doesn't) -- PARTITION BY:
rank within each group
SELECT city, customer_id, annual_spend, RANK() OVER (PARTITION BY city ORDER
BY annual_spend DESC) AS city_rank FROM customers; -- Each city gets its own 1,2,3... ranking --
Top 3 per category
SELECT category, product_name, price, rnk FROM ( SELECT category, product_name,
price, RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rnk FROM products ) ranked WHERE
rnk <= 3; -- NTILE: divide into N equal buckets (quartiles, deciles)
SELECT customer_id,
annual_spend, NTILE(4) OVER (ORDER BY annual_spend DESC) AS quartile, -- Q1=top 25% NTILE(10) OVER
(ORDER BY annual_spend DESC) AS decile FROM customers;
```

## SECTION 3

# Aggregate Window Functions

```
-- RUNNING TOTAL (cumulative sum)
SELECT order_date, total_amount, SUM(total_amount) OVER (ORDER BY
order_date) AS running_total FROM orders ORDER BY order_date; -- Running total WITHIN each customer
SELECT customer_id, order_date, total_amount, SUM(total_amount) OVER ( PARTITION BY customer_id
ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW ) AS customer_running_total
FROM orders; -- MOVING AVERAGE (7-day)
SELECT order_date, SUM(total_amount) AS daily_revenue,
AVG(SUM(total_amount)) OVER ( ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW --
current + 6 previous = 7 days ) AS moving_avg_7d FROM orders GROUP BY order_date; -- Percentage of
total
SELECT city, SUM(annual_spend) AS city_revenue, SUM(SUM(annual_spend)) OVER () AS
grand_total, ROUND(100.0 * SUM(annual_spend) / SUM(SUM(annual_spend)) OVER (), 2) AS pct_of_total
FROM customers GROUP BY city;
```

## SECTION 4

# LAG, LEAD, FIRST\_VALUE, LAST\_VALUE

```
-- LAG: access previous row's value -- LEAD: access next row's value
SELECT order_date,
SUM(total_amount) AS revenue, LAG(SUM(total_amount)) OVER (ORDER BY order_date) AS
prev_day_revenue, LEAD(SUM(total_amount)) OVER (ORDER BY order_date) AS next_day_revenue, -- Month
over month change
SUM(total_amount) - LAG(SUM(total_amount)) OVER (ORDER BY order_date) AS
day_over_day_change, -- Percentage change
ROUND(100.0 * (SUM(total_amount) - LAG(SUM(total_amount)))
```

```
OVER (ORDER BY order_date)) / NULLIF(LAG(SUM(total_amount)) OVER (ORDER BY order_date), 0), 2) AS
pct_change FROM orders GROUP BY order_date ORDER BY order_date; -- LAG with offset and default
SELECT month, revenue, LAG(revenue, 1, 0) OVER (ORDER BY month) AS prev_month, -- 1 row back
LAG(revenue, 12, 0) OVER (ORDER BY month) AS same_month_last_year -- 12 rows back FROM
monthly_revenue; -- FIRST_VALUE and LAST_VALUE SELECT customer_id, order_date, total_amount,
FIRST_VALUE(total_amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS
first_order_amount, LAST_VALUE(total_amount) OVER ( PARTITION BY customer_id ORDER BY order_date
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING ) AS last_order_amount -- need explicit
frame for LAST_VALUE! FROM orders;
```

## ■ PRACTICE Q & A

**Q1.** What is the difference between window functions and GROUP BY?

*A: GROUP BY collapses rows -- you get one output row per group. Window functions keep ALL rows -- each row gets a computed value calculated across a defined window of rows. Use GROUP BY for summaries, window functions for row-level analytics.*

**Q2.** What is the difference between RANK and DENSE\_RANK?

*A: Both give same rank to ties. RANK then skips numbers (1,2,2,4), DENSE\_RANK never skips (1,2,2,3). Use DENSE\_RANK when you want continuous ranking, RANK when gaps indicate positions skipped.*

**Q3.** What does PARTITION BY do in a window function?

*A: Divides rows into groups for the calculation, similar to GROUP BY but without collapsing rows. RANK() OVER (PARTITION BY city ORDER BY spend DESC) gives each city its own 1,2,3 ranking independently.*

**Q4.** How do you calculate month-over-month growth?

*A: Use LAG to get previous month value, then (current - previous) / previous \* 100. LAG(revenue, 1) OVER (ORDER BY month) gets the prior month. Wrap denominator in NULLIF to avoid divide-by-zero.*

**Q5.** Why does LAST\_VALUE often give unexpected results?

*A: LAST\_VALUE defaults to ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- it returns the last row seen so far, not the actual last row in the partition. Fix: add ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.*

| ■ Window Functions Cheat Sheet                                        |                              |
|-----------------------------------------------------------------------|------------------------------|
| <code>ROW_NUMBER() OVER (ORDER BY col)</code>                         | Unique sequential row number |
| <code>RANK() OVER (ORDER BY col)</code>                               | Rank with gaps on ties       |
| <code>DENSE_RANK() OVER (ORDER BY col)</code>                         | Rank without gaps            |
| <code>NTILE(4) OVER (ORDER BY col)</code>                             | Divide into quartiles        |
| <code>SUM(col) OVER (ORDER BY col)</code>                             | Running total                |
| <code>AVG(col) OVER (ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)</code> | 7-day moving average         |
| <code>PARTITION BY grp ORDER BY col</code>                            | Calculate within groups      |
| <code>LAG(col, 1) OVER (ORDER BY col)</code>                          | Previous row value           |
| <code>LEAD(col, 1) OVER (ORDER BY col)</code>                         | Next row value               |
| <code>FIRST_VALUE(col) OVER (...)</code>                              | First value in window        |
| <code>SUM(col) OVER ()</code>                                         | Grand total in denominator   |
| <code>ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING</code> | Full partition frame         |